

Project Contributors — Team Task Assignments & Presentation Prep

Team Overview

This project is a Student Record System in C. The team is divided into various roles. Each pair of persons implements the same function; the best version is selected. All persons review the group leader’s work to understand the architecture. Two shared reviewers per task pair ensure quality.

Task Assignments

Table 1: Team Task Assignments and Reviewers

Person	Task	Function	Reviewers
1	Group Leader	Architecture and core building blocks: main (CLI orchestration, menu loop, confirmation prompts, memory cleanup, consume trailing stdin, allocate array), data model structs (Student, StudentRecordSystem), displayMenu, displayWelcomeMessage, clearInputBuffer, initializeSystem, confirmAction	Everyone re-views
2, 3	Add Student	addStudent (input validation, duplicate detection, resize)	Persons 4, 5
4, 5	Display All Students	displayAllStudents (format table, compute PASS/FAIL)	Persons 6, 7
6, 7	Search Student	searchStudent (locate, display, not-found handling)	Persons 8, 9
8, 9	Modify Student	modifyStudent (find, partial updates, preserve blanks)	Persons 10, 11
10, 11	Remove Student	removeStudent (locate, shift array, decrement count)	Persons 12, 2
12, 2*	Calculate Average	calculateAverageMarks (sum, average, display)	Persons 3*, 4*
3*, 4*	Sort Students	sortStudents, compareMarks (qsort, ascending/descending)	Persons 5*, 6*
5*, 6*	Save to File	saveToFile (CSV export, headers, pass/fail status)	Persons 8*, 7*
7*, 8*	Load from File	loadFromFile (CSV parsing, array resize, data population)	Persons 9*, 10*
*Alt task (fallback if primary task completed early)			

Team Process

We organised the work in **phases**, so that functions which are more independent were designed and implemented first, and functions that depend on others came later. [Image of Project Workflow Phased Approach]

1. Phase 0 – Leader skeleton and core utilities
- Person 1 (leader) defines the data structures and core utilities that everything else depends on: Student, StudentRecordSystem, initializeSystem, main, displayMenu, displayWelcomeMessage, clearInputBuffer, and confirmAction.
2. Phase 1 – Core data creation
- addStudent (validated inserts, duplicate check, capacity checks).
3. Phase 2 – Read-only features on created data
- Once addStudent exists and we can store records, we implement the functions that **read** and present those records:
 - displayAllStudents (table formatting and PASS/FAIL display).

- `searchStudent` (find by roll number).

4. Phase 3 – Write operations

- Functions that **change** the in-memory array are implemented next:
 - `modifyStudent` (editing an existing record).
 - `removeStudent` (shifting and shrinking the logical count).

5. Phase 3 – Calculations and sorting

- Once we can add, view, and remove students, we implement:
 - `calculateAverageMarks` (sum and average).
 - `sortStudents/compareMarks` (ascending/descending order using `qsort`).

6. Phase 4 – File operations

- Finally, we add functions that depend on a stable in-memory representation but extend it with storage:
 - `saveToFile` (writing comma-separated lines to a text file).
 - `loadFromFile` (rebuilding the in-memory list from the file).

Final Documentation Structure

The comprehensive final documentation will follow this structure:

1. **Table of Contents** — all sections with page numbers
2. **Group Details** — group number, member names, student ID
3. **Requirements** — original problem statement
4. **Planning and Considerations** — modular approach, data model, code review strategy
5. **Program Flow and User Story** — Program Flow diagram, how the program runs (User Story) [Image of Basic C Program Execution Flow Diagram]
6. **Program/Code** — full `student_record.c` code
7. **Quick Code Explanation** — 1–2 paragraphs per function: what it does, design choices, edge case handled
8. **Conclusion**

Preparation Instructions

For All Team Members:

- **Read** `c_tutorial.MD` (if new to C), then `student_record_commented.c` (learn structure).
- **Study** `student_record.c` carefully.
- **For your assigned task:** Read both implementations (yours and your pair's), understand the differences.
- **Prepare ONE slide** covering:
 - Function name and what it does
 - Why the design choices were made (safety, efficiency, user experience)
 - One edge case it handles well
 - A short demo command (e.g., "I will run option 5 to remove a student")
- **Test your demo:** Compile with `gcc -o student_record student_record.c`, run your function, time it (10–30s).
- **Save your slide** as `presentation/person_<N>_task.pdf` or `.pptx`.
- Read `HOW_TO_PREPARE_FOR_PRESENTATION.md` for more details on how to prepare for the presentation.
- Read `PRESENTATION_QA.md` for like questions and answer that may be asked

For Reviewers:

- Read the two paired implementations carefully.
- Write or run one simple test to verify correctness.
- Check for input validation, memory safety, edge cases.
- Provide feedback to your pair.

For Person 1 (Group Leader):

- Ensure all final implementations are integrated and tested.
- Coordinate team presentations and demo rehearsal.
- Present the architecture and integrated demo first (add → load → display → sort → save → exit).

How We All Worked Together

The team followed a **modular, pair-based approach**: the group leader established the data model and main flow. Each pair independently implemented the same function; two shared reviewers compared both versions and selected the best based on quality and correctness. Everyone studied the leader’s code to understand integration. All implementations were merged into a single, cohesive `student_record.c`. Each person prepared a presentation slide and demo to explain their function’s design choices and edge cases, ensuring every team member contributed meaningfully to both code and presentation.

Sample Leader Code Skeleton (Starting Point)

Below is a simplified version of the **group leader’s starting code** that everyone used as a reference. It shows the core structures, basic helper functions (`displayMenu`, `displayWelcomeMessage`, `clearInputBuffer`, `confirmAction`), function declarations, and the main menu loop where each person plugs in their own function implementation.

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #include <ctype.h>
5
6 #define MAX_NAME_LENGTH 50
7 #define PASSING_MARKS 40
8
9 // Data model
10 typedef struct {
11     char name[50];
12     int rollNumber;
13     float marks;
14 } Student;
15
16 typedef struct {
17     Student *students;
18     int count;
19     int capacity;
20 } StudentRecordSystem;
21
22 // Function declarations (everyone implements one or more of these)
23 void initializeSystem(StudentRecordSystem *records);
24 void displayWelcomeMessage();
25 void displayMenu();
26 void addStudent(StudentRecordSystem *records);
27 void displayAllStudents(const StudentRecordSystem *records);
28 void searchStudent(const StudentRecordSystem *records);
29 void modifyStudent(StudentRecordSystem *records);
30 void removeStudent(StudentRecordSystem *records);
31 void calculateAverageMarks(const StudentRecordSystem *records);
32 void sortStudents(StudentRecordSystem *records, int ascending);
33 void saveToFile(const StudentRecordSystem *records);
34 void loadFromFile(StudentRecordSystem *records);
35 int confirmAction(const char *actionDescription);
36 void clearInputBuffer();
37
38 int main(void) {
39     StudentRecordSystem records;
40     initializeSystem(&records);
41
42     char userName[50];
43     displayWelcomeMessage();
44     printf("\nPlease enter your name: ");
45     fgets(userName, sizeof(userName), stdin);
46     userName[strcspn(userName, "\n")] = '\0';
47
48     printf("\nHello, %s! Welcome to the Student Record System.\n\n", userName);
49
50     int choice;
51     do {
52         displayMenu();
53         printf("Enter your choice (1-11): ");
54
55         if (scanf("%d", &choice) != 1) {
56             printf("Invalid input! Please enter a number.\n");
```

```

57         clearInputBuffer();
58         continue;
59     }
60     clearInputBuffer();
61
62     switch (choice) {
63     case 1:
64         if (!confirmAction("add a new student")) break;
65         addStudent(&records);
66         break;
67     case 2:
68         if (!confirmAction("display all students")) break;
69         displayAllStudents(&records);
70         break;
71     case 3:
72         if (!confirmAction("search for a student")) break;
73         searchStudent(&records);
74         break;
75     case 4:
76         if (!confirmAction("modify student records")) break;
77         modifyStudent(&records);
78         break;
79     case 5:
80         if (!confirmAction("remove a student")) break;
81         removeStudent(&records);
82         break;
83     case 6:
84         if (!confirmAction("calculate average marks")) break;
85         calculateAverageMarks(&records);
86         break;
87     case 7:
88         if (!confirmAction("sort students by marks in ascending order")) break;
89         sortStudents(&records, 1);
90         break;
91     case 8:
92         if (!confirmAction("sort students by marks in descending order")) break;
93         sortStudents(&records, 0);
94         break;
95     case 9:
96         if (!confirmAction("save records to a file")) break;
97         saveToFile(&records);
98         break;
99     case 10:
100        if (!confirmAction("load records from a file")) break;
101        loadFromFile(&records);
102        break;
103    case 11:
104        if (confirmAction("exit the program")) {
105            printf("Thank you for using the Student Record System, %s! Goodbye!\n", userName);
106        } else {
107            choice = 0; // stay in menu
108        }
109        break;
110    default:
111        printf("Invalid choice! Please try again.\n");
112    }
113 } while (choice != 11);
114
115 free(records.students);
116 return 0;
117 }
118
119 void initializeSystem(StudentRecordSystem *records) {
120
121     // Choose a small default capacity; realloc will grow if needed.
122     records->capacity = 100;
123     records->students = (Student*)malloc(records->capacity * sizeof(Student));
124     records->count = 0;
125
126     // Simple check: exit if heap allocation fails
127     if (records->students == NULL) {
128         printf("Memory allocation failed!\n");
129         exit(1);
130     }
131 }
132
133 void displayWelcomeMessage() {
134     printf("=====\n");
135     printf("      STUDENT RECORD SYSTEM\n");
136     printf("=====\n");
137 }
138
139 void displayMenu() {
140     printf("=====\n");
141     printf("      MAIN MENU\n");
142     printf("=====\n");
143     printf("1. Add Student\n");
144     printf("2. Display All Students\n");
145     printf("3. Search Student by Roll Number\n");
146     printf("4. Modify Student Record\n");
147     printf("5. Remove Student\n");
148     printf("6. Calculate Average Marks\n");
149     printf("7. Sort Students by Marks (Ascending)\n");
150     printf("8. Sort Students by Marks (Descending)\n");
151     printf("9. Save Records to a Comma-Separated Text File (CSV-Style)\n");

```

```

152     printf("10. Load Records from a Comma-Separated Text File (CSV-Style)\n");
153     printf("11. Exit\n");
154     printf("-----\n");
155 }
156
157 int confirmAction(const char *actionDescription) {
158     char buffer[16];
159     printf("Are you sure you want to %s? Press y/Y to continue, n/N to cancel: ", actionDescription);
160     if (fgets(buffer, sizeof(buffer), stdin) == NULL) {
161         printf("Action cancelled.\n");
162         return 0;
163     }
164     if (buffer[0] == 'y' || buffer[0] == 'Y') {
165         return 1;
166     }
167     printf("Action cancelled.\n");
168     return 0;
169 }
170
171 void clearInputBuffer() {
172     int c;
173     while ((c = getchar()) != '\n' && c != EOF) {
174         /* discard extra characters */
175     }
176 }

```

Sample Leader Code Skeleton (C)

Implementation Plan for Group Members

To keep everyone aligned, the group leader proposed this implementation plan:

1. Understand the skeleton

- Every member first reads the leader's skeleton and the fully commented file `student_record_commented.c` to understand:
 - Data structures (`Student`, `StudentRecordSystem`).
 - The menu loop and how functions are called.
 - How confirmation (`confirmAction`) and input handling work.

2. Implement your assigned function in isolation

- Each pair (e.g., Persons 4 and 5 for `addStudent`) copies the function signature from the skeleton and implements it inside their own local file or branch.
- They focus on:
 - Correct logic.
 - Input validation and error messages.
 - Handling edge cases listed in the documentation.

3. Use the common main for testing

- All contributors compile and run using the **same** `main` (the skeleton) so behavior is consistent.
- Example: compile with `gcc -o student_record student_record.c` and test their menu option.

4. Review and compare implementations

- The assigned reviewers for each pair run both versions of the function.
- They check:
 - Does it handle invalid input safely?
 - Does it match the behavior described in the documentation?
 - Is the code clear and well-structured?

5. Select and integrate the best version

- Reviewers and the group leader agree on which version to integrate.
- The chosen implementation is merged into the main `student_record.c` in the appropriate function body.

6. Final integration pass by the leader

- The leader ensures:
 - All functions compile together.
 - There are no duplicate or unused functions.
 - Menu options, confirmation prompts, and file paths are consistent.

7. Presentation preparation

- Each member prepares a 1–2 minute explanation of **their** function:
 - What it does.
 - One design decision (e.g., why we used `fgets`, why we check range 0–100, why we used `qsort`).

- One edge case it handles.
 - Everyone tests their menu option with the **confirmation prompts** turned on so they know how the program behaves when the user chooses "yes" or "no".
 - The leader presents the overall architecture and runs a full demo (add → load → display → search/modify/remove → sort → save/load → exit) including at least one cancelled action to show the confirmation dialog in action.
 - Read `HOW_TO_PREPARE_FOR_PRESENTATION.md` for more details on how to prepare for the presentation.
 - Read `Presentation_QA.md` for like questions and answer that may be asked
-

Contacts

- **Person 1 (Leader):** Architecture, integration, rehearsal coordination.
- **Your 2 Shared Reviewers:** Code quality, testing, demo validation.
- **Everyone:** Prepare slides, study the code, participate in rehearsal.

Goal: Make it clear to the lecturer that every person contributed to design, implementation review, and presentation quality.